

ELEC 475 Lab 2, Alistair Barfoot and Luke Barry

Contents

1	Network Architecture	2
2	DataLoader and Dataset	2
3	Hyperparameters	3
4	Augmentation	3
5	Results	3
6	Performance	4
7	Ensemble Approach	5
8	LLM Usage	6

1 Network Architecture

SnoutNet is a Convolutional Neural Network designed for regressing the (x,y) coordinates of an animal's nose from a RGB image. It takes an input image of 227x227x3 and outputs a 2D coordinate vector. It is comprised of three convolutional blocks each followed by a ReLU activation and a Max Pooling layer, then three fully connected (FC) layers.

Layer	Input Shape	Output Size	Parameters
Conv1	227x227x3	114x114x64	in_channels=3, out_channels=64, kernel_size=3, stride=2, padding=1
ReLU1	114x114x64	114x114x64	
MaxPool1	114x114x64	57x57x64	kernel_size=3, stride=2, padding=1
Conv2	57x57x64	29x29x128	in_channels=64, out_channels=128, kernel_size=3, stride=2, padding=1
ReLU2	29x29x128	29x29x128	
MaxPool2	29x29x128	15x15x128	kernel_size=3, stride=2, padding=1
Conv3	15x15x128	8x8x256	in_channels=128, out_channels=256, kernel_size=3, stride=2, padding=1
ReLU3	8x8x256	8x8x256	
MaxPool3	8x8x256	4x4x256	kernel_size=3, stride=2, padding=1
FC1	4x4x256(4096)	1024	
ReLU4	1024	1024	
FC2	1024	1024	
ReLU5	1024	1024	
FC3	1024	2 (u,v)	

2 DataLoader and Dataset

The images and labels are loaded through a custom pytorch dataset class called Custom-Dataset, which reads the annotation file train-noses.txt containing image file names and their corresponding coordinates to their noses. For each entry the dataset loads the image and converts it to an RGB format and parses the coordinate string to numerical values. Images are resized to 227x227 pixels and the transformed nose coordinates are returned as a vector.

A factcheck was performed by creating a small test script that loads a sample and prints the shapes and values. Throughout each train an `example_debug.png` image will appear one in every thousand images with the image and its corresponding snout vector. This ensures that the dataset is loading correctly each time with the augmentation applied.

3 Hyperparameters

The SnoutNet model was trained for 50 epochs with a batch size of 32 using the Adam optimizer and Kaiming Normal weight initialization. The learning rate scheduler is applied to monitor validation loss and reduces the learning rate by a factor of 2 if no improvement has been observed in 3 consecutive epochs. The learning rate was set to 3×10^{-4} and a weight decay of 3×10^{-6} .

The loss function used was the Mean Squared Error (MSE) loss. Early stopping is also implemented if no validation loss improvement has been observed for 5 epochs. In the case of early stopping, the model weights are reverted to the best observed validation loss.

4 Augmentation

Data augmentation was optionally applied to the training. A horizontal flip to simulate mirrored images was applied as well as a random rotation of $\pm 10^\circ$ to provide rotational variance. For both horizontal flip and a random rotation, the label for the nose vector needed to be adjusted, since the images position within the frame changed the snout of each animal would have changed in position too.

The horizontal flip occurred with a probability of 0.1, and the random rotation was applied with a probability of 1.

5 Results

The training was conducted on a NVIDIA RTX 3060 Super laptop edition. To calculate the distance from ground truth for testing, the distances were adjusted to account for the resizing of the images to 227x227 pixels and rounded to the nearest integer.

The training results for each model and augmentation strategy are summarized in Table 1, while the error statistics are presented in Table 2.

Model	Augmentation	Epochs	Train Loss	Test Loss	Time Elapsed (h:mm)
SnoutNet	None	26	213	398	0:13
	Flip	26	347	541	0:14
	Rotate	33	291	455	0:18
	Flip / Rotate	50	395	427	0:27
SnoutNet-A	None	14	87	168	0:10
	Flip	16	72	182	0:11
	Rotate	34	55	119	0:24
	Flip/Rotate	43	51	103	0:31
SnoutNet-V	None	6	94	147	0:46
	Flip	9	118	123	1:09
	Rotate	20	35	87	2:34
	Flip/Rotate	14	131	129	1:48

Table 1: Training results for each model and augmentation strategy.

Model	Augmentation	Localization Error				Localization Error (4 Worst)				Localization Error (4 Best)			
		min	max	mean	stdev	min	max	mean	stdev	min	max	mean	stdev
SnoutNet	None	1	145	26	19.3	106	145	122	16.4	1	2	1	0.4
	Flip	1	132	26	19.2	109	132	119	9.3	1	2	1	0.4
	Rotate	1	171	24	19.5	113	171	136	22.1	1	1	1	0.2
	Flip/Rotate	0	140	23	18.1	103	140	123	13.7	0	2	1	0.6
SnoutNet-A	None	0	117	14	13.4	82	117	94	13.5	0	1	1	0.1
	Flip	0	99	13	12.5	73	99	88	10.0	0	1	1	0.1
	Rotate	0	111	10	12.7	71	111	93	14.5	0	1	0	0.2
	Flip/Rotate	0	116	10	11.1	68	116	85	18.4	0	0	0	0.2
SnoutNet-V	None	0	107	10	13.3	84	107	96	9.7	0	0	0	0.2
	Flip	0	88	10	9.3	71	88	82	6.3	0	1	1	0.2
	Rotate	1	120	9	10.0	80	120	93	15.7	1	1	1	0.1
	Flip/Rotate	0	110	9	11.1	84	110	94	9.8	0	1	0	0.1
Ensemble													

Table 2: Localization error statistics for each model and augmentation strategy.

6 Performance

Overall, the model performed well in identifying the snout position of the animals in the images. The best performing (non-ensemble) model was SnoutNet-V trained with rotation augmentation, achieving a mean localization error of 9 pixels with a standard deviation of 10.0 pixels. This model achieved a test loss of 87 while training and reached 20 epochs before early stopping.

In general, the flip augmentation did not significantly improve performance, and in the case of SnoutNet-V, it actually led to a slight decrease in performance when going from rotation only to flip/rotate.

The SnoutNet architecture was the worst-performing model overall when compared to the SnoutNet-A and SnoutNet-V variants. This was expected, as the latter two models have much more parameters and more advanced techniques such as dropout. The SnoutNet-A model performed better than the base model, but just slightly worse than SnoutNet-V despite the significant difference in model size and training time.

7 Ensemble Approach

The ensemble approach combined the predictions from each of the three models by asking each model to predict the snout position for a given image, then averaging the (u,v) coordinates from each model to arrive at a final prediction.

8 LLM Usage

Link to LLM Conversation

For this lab, GitHub Copilot was used to assist in generating the structure of the SnoutNet dataset and DataLoader classes. However, the code generated by the LLM was not just copied and ran in the project. Instead, each line of the code generated by the LLM was carefully reviewed and tested to ensure that it functioned as intended.

Debugging files and lines were added at every step of the process to ensure that each part was functioning correctly and as predicted. In the final version of the lab, much of the code generated by the LLM has been modified or removed entirely as it was not taken just at face value.